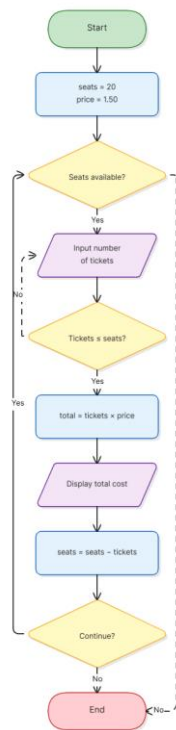


Task 1: Algorithm Design.....	1
Task 2: Logic Flowchart.....	1
Task 3: Python Code	1
Success Case	2
Not Enough Seats Case	3

Task 1: Algorithm Design

1. Start the program.
2. Set the total number of available seats to 20.
3. Set the ticket price to \$1.50.
4. Repeat the process while there are seats available.
5. Ask the operator to enter the number of tickets needed.
6. Check if the requested tickets are less than or equal to available seats.
7. If yes, calculate total cost (tickets × price).
8. Display the total cost to the operator.
9. Subtract the booked tickets from available seats.
10. Ask the operator if they want to continue or stop the system.
11. If no seats are left or the operator chooses to stop, end the program.

Task 2: Logic Flowchart



Task 3: Python Code

```
1  # GreenCampus Shuttle Digital Ticket Kiosk
2
3  # Step 1: Initialize variables
4  available_seats = 20
5  ticket_price = 1.50
6
7  print("=== GreenCampus Shuttle Ticket System ===")
8
9  # Step 2: Loop until full or user exits
10 while available_seats > 0:
11     print("\nAvailable seats:", available_seats)
12
13     # Step 3: Get input
14     try:
15         tickets = int(input("Enter number of tickets: "))
16     except:
17         print("Invalid input! Please enter a number.")
18         continue
19
20     # Step 4: Check availability
21     if tickets <= available_seats and tickets > 0:
22
23         # Step 5: Calculate cost
24         total_cost = tickets * ticket_price
25
26         # Step 6: Output result
27         print("Total cost: $", total_cost)
28
29     # Step 7: Update seats
30     available_seats -= tickets
31
32     print("Booking successful!")
33
34     else:
35         print("Not enough seats or invalid number.")
36
37     # Step 8: Continue or stop
38     choice = input("Continue? (yes/no): ").lower()
39     if choice != "yes":
40         break
41
42 # Step 9: End message
43 if available_seats == 0:
44     print("\nShuttle is FULL.")
45 else:
46     print("\nSystem stopped by operator.")
47
48 print("=== Thank you ===")
```

Success Case

When the number of tickets entered by the operator is **less than or equal to the available seats**, the system allows the booking.

The program then calculates the total cost by multiplying the number of tickets by the ticket price (\$1.50). After that, it displays the total cost and **reduces the available seats** using `available_seats -= tickets`.

This means the booking is completed successfully and the system is updated for the next student.

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

System stopped by operator.
=== Thank you ===
PS C:\Users\DELL\Documents\Foundation\test> & C:\Users\DELL\AppData\Local\Programs\Python\Python314\python.exe c:/Users/DELL/Documents/Foundation/test/test.py
=== GreenCampus Shuttle Ticket System ===

Available seats: 20
Enter number of tickets: 6
Total cost: $ 9.0
Booking successful!
Continue? (yes/no): y

System stopped by operator.
=== Thank you ===
PS C:\Users\DELL\Documents\Foundation\test> |
```

Not Enough Seats Case

When the number of tickets entered is **greater than the available seats**, the system does not allow the booking.

Instead, it shows an error message like **"Not enough seats available"**. The number of seats remains the same because no booking is made.

This ensures that the system does not overbook the shuttle and keeps the seat count accurate.

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\Users\DELL\Documents\Foundation\test> & C:\Users\DELL\AppData\Local\Programs\Python\Python314\python.exe c:/Users/DELL/Documents/Foundation/test/test.py  
=== GreenCampus Shuttle Ticket System ===
```

```
Available seats: 20  
Enter number of tickets: 21  
Not enough seats or invalid number.  
Continue? (yes/no): y
```

```
System stopped by operator.  
=== Thank you ===
```

```
PS C:\Users\DELL\Documents\Foundation\test> █
```